SISTEMAS COMPUTACIONAIS AVANÇADOS (SISTCA)

ADVANCED COMPUTING SYSTEMS

LICENCIATURA EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO
POLITÉCNICO DO PORTO

Lab Classes Script:

Introduction to Docker



Version Control

Version number	Date issued	Authors	Update information
V1.0		Vítor Barbosa (1221412) José Leal (1220971) João Fonseca (1220918) Simão Seixas (1201473) Supervision: Prof. Paula Viana	Original version of the script.

Table of Contents

1. Introduction 1

1.1 Scientific/Technological Context 1

1.2 Motivation for this topic/lab script 1

1.3 Objectives 1

1.4 Structure 1

2. Theoretical (scientific/technological background) 2

2.1 Main theoretical concepts/terminology 2

2.2 State-of-The-Art 3

2.3 Containers vs VMs 4

2.4 Virtual Machines (VMs) 5

2.5 Advantages 5

2.6 Disadvantages 5

2.7 Docker Containers 5

2.8 Advantages 5

2.9 Disadvantages 6

2.10 Key Differences 6

2.11 Target Audience 6

2.12 Security with Docker 7

3. Outlook of the technology 9

3.1 Setup/Installation (SW/HW) 9

3.1.1 Linux 9

3.1.2 Windows 10

3.1.3 Mac 12
3.2 First steps 13
4. Tutorial/Functionality 16
4.1 Docker Compose / Multi Container APPs 16
4.2 Structure of a docker-compose.yml File 16
4.3 Advantages of Docker Compose 17
4.4 Docker hub 17
4.5 Docker CLI 18
4.6 Docker desktop 18
5. Exercise(s) 18
5.1 Problem A 18
5.1.1 (Re)solution A 19
5.2 Problem B 22
5.2.1 (Re)solution B 22
6. Challenge 24
7. References 26

1. Introduction

1.1 Scientific/Technological Context

For some time, software developers used Virtual Machines (VM's) to perform different specific tasks, but they require their own Operating System (OS), their own kernel, drivers, etc. With the arrival of containers, we can use them for the same purposes but requiring less recourses because they run like a process and share the same kernel. Docker is a platform that lets you create and manage containers where you can run your applications, in a fast, easy and lightweight way.

1.2 Motivation for this topic/lab script

With Docker being used more and more by developers, its importance has skyrocketed in recent years. Due to its ease of use, Docker allows developers to run applications effortlessly while ensuring consistency across different environments. It enables teams to work with the same version of an application, reducing compatibility issues and streamlining deployment. Given its growing significance in modern software development, we decided to do a Docker lab script to provide hands-on experience with Docker, helping users understand its core concepts and practical applications.

1.3 Objectives

This work fundamentally aims to provide knowledge about container technology, particularly Docker, from its theoretical foundations to practical applications.

By the end of this report and following the proposed steps, are expected to have a basic understanding of this technology and be able to apply it in the intended projects.

1.4 Structure

This guide is organized into six main sections that together provide a comprehensive exploration of docker, and all the technologies related to it.

It begins with an introduction, where the motivation for the study is detailed, clear objectives are outlined.

The Theoretical Background follows, offering a deep dive into key concepts, state-of-the-art developments, target audience considerations, architectural design,

security measures, and an explanation of the differences between containers and virtual machines.

The third section contains the proper outlook of the technology, where the reader is going to be guided in the setup and installation of the necessary software and hardware across different operating systems. As well as a detailed introduction of basic Docker command-line operations.

Next, the Tutorial/Functionality section provides hands-on examples using Docker Compose for multi-container applications, alongside explorations of Docker Desktop, Docker Hub, and the Docker CLI.

The Exercises section is designed to reinforce learning through practical problem-solving, featuring specific challenges and their solutions, such as running docker applications.

Finally, the paper concludes with a Challenge that pushes the readers to understand by addressing an advanced practical exercise with Docker.

2. Theoretical (scientific/technological background)

2.1 Main theoretical concepts/terminology

When we analyze Docker, we need to understand its architecture, and Docker utilizes a "client-server architecture" (Docker, s.d.). The client is where you will type your commands like *docker run ,* and it will send those commands to the Docker daemon (the server side of the connection) where it will run and execute those commands. Both client and daemon can be on the same system, or on different ones, in our case we will run both in the same system (our computer). The client and the daemon communicate "using a REST API, over UNIX sockets or a network interface" (Docker, s.d.).

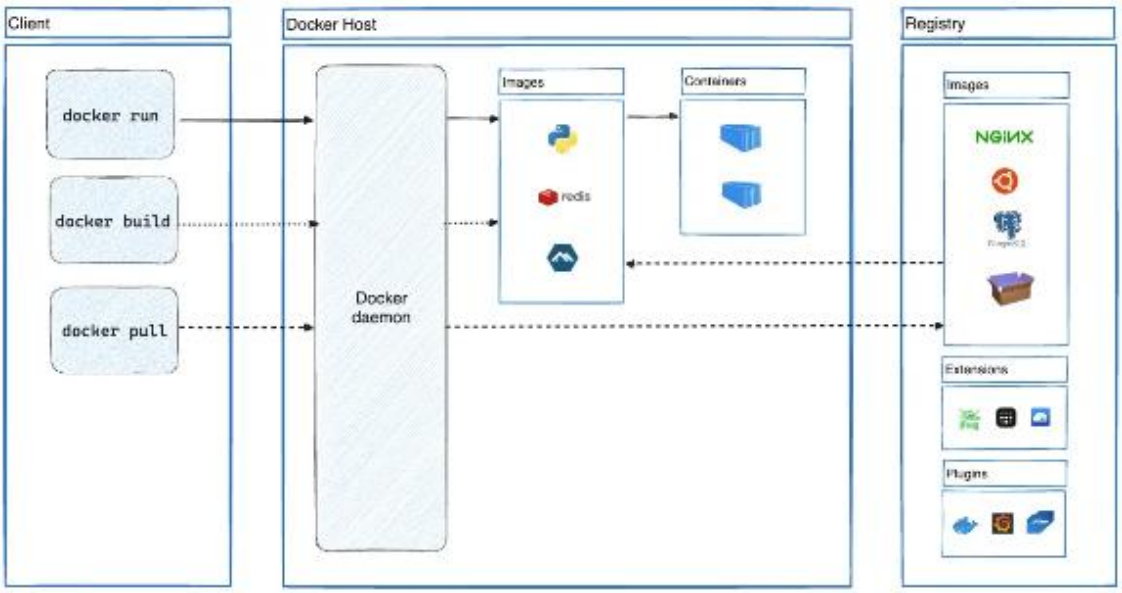


Figure 1 Docker Architecture

Another concept that we need to learn is Containers. Containers are "isolated processes for each of your app's components" (Docker, s.d.), this means that you can have different pieces of software running at the same time, sharing resources at the same system and isolated from each other. These containers have everything they need to work on their own, so, they need some files and configurations to be able to function. For that we, have Images which is "a standardized package that includes all of the files, binaries, libraries, and configurations to run a container." (Docker, s.d.).

In the Figure 1 the Docker Host communicating with a Registry, that is a "centralized location" (Docker, s.d.) that stores Docker Images so that they can be shared among other users. Registries also have Extensions and Plugins that are additional features for Docker, some may be third-party, and others built by Docker.

Docker is written in Go, that is an open-source programming language that is supported by Google. You can learn more about Go in https://go.dev/.

2.2 State-of-The-Art

While Docker is a powerful tool, it also has some limitations. One of the main challenges is performance overhead, as Docker, while lightweight compared to virtual machines, can still introduce some inefficiencies, especially for applications requiring direct access to hardware. Another limitation is limited support for GUI applications,

as Docker is primarily designed for command-line and backend services, making it difficult to run graphical applications without additional setup.

Security is also a concern, as containers share the host OS kernel, meaning vulnerabilities in one container or misconfigurations can pose risks to the entire system. Additionally, persistent storage can be tricky to manage, as data needs to be carefully configured to persist across container restarts and migrations.

Networking can also be complex, as Docker's networking model requires additional configurations to work with multi-container applications or existing infrastructures. Compatibility issues can arise, especially on Windows, where Docker runs using a virtualized Linux kernel, leading to restrictions on some low-level OS features.

Finally, Docker has a learning curve, particularly when working with container orchestration tools like Kubernetes. While Docker simplifies deployment, mastering its advanced features and configurations can take time. [1]

2.3 Containers vs VMs

Docker containers and virtual machines (VMs) are both technologies that enable virtualization, allowing multiple isolated environments to run on a single physical machine. However, they differ significantly in their architecture, resource utilization, performance, and use cases. [16]

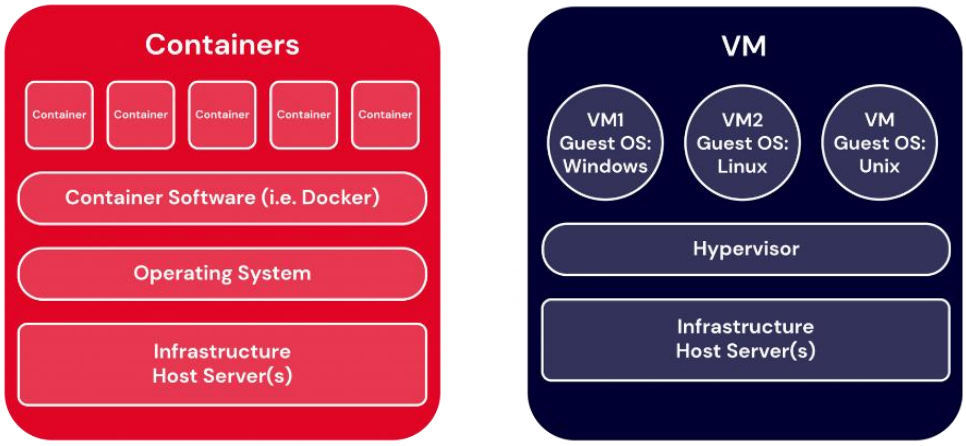


Figure 2 Architecture of containers and VMs

2.4 Virtual Machines (VMs)

VMs emulate entire physical computers, including the hardware and operating system (OS). They run on a hypervisor, which manages the distribution of physical resources to each VM. This setup allows multiple VMs, each with their own OS, to operate on a single physical machine.

2.5 Advantages

Full Isolation: Each VM operates independently with its own OS, providing strong isolation between environments. This isolation enhances security, as issues in one VM are less likely to affect others.

Diverse OS Support: VMs can run different operating systems on the same hardware. For example, you can have Windows and Linux VMs co-existing on a single host, which is beneficial for testing applications across various platforms

2.6 Disadvantages

Resource Intensive: Running multiple VMs requires significant system resources, as each VM includes a full OS. This can lead to higher overhead and reduced performance compared to containers.

Slower Boot Times: Booting up a VM can take several minutes, as it involves starting a complete operating system.

2.7 Docker Containers

Containers, on the other hand, virtualize the operating system rather than the hardware. They package applications and their dependencies into isolated units that share the host OS kernel but run in separate user spaces. This approach makes containers lightweight and efficient.

2.8 Advantages

Lightweight: Containers share the host OS kernel, eliminating the need for separate OS instances. This results in lower overhead and allows for running more containers on the same hardware.

Fast Deployment: Containers can start in seconds, as they don't require booting an OS. This rapid deployment is advantageous for scaling applications quickly.

Portability: Since containers encapsulate the application and its dependencies, they can run consistently across different environments, from a developer's laptop to production servers.

2.9 Disadvantages

Shared Kernel: Containers share the host OS kernel, which can lead to security concerns. If a container is compromised, there's a potential risk to other containers and the host system.

Less Isolation: While containers provide process and file system isolation, they don't offer the same level of isolation as VMs. This can be a consideration for applications requiring stringent security measures.

2.10 Key Differences

We can visualize in the next table a summary of the main differences between Virtual machines and Docker Containers.

Table 1 Virtual Machines and Containers aspects

Aspect	Virtual Machines	Containers
Architecture	Virtualizes hardware, runs separate OS instances	Virtualizes OS, shares host OS kernel
Resource Usage	More resource-intensive	More efficient, fewer resources
Performance	May experience degradation	Near-native performance
Isolation	Stronger isolation with separate OS	Process-level isolation
Best Use Cases	Full OS functionality, strong isolation needs	Lightweight, scalable applications

2.11 Target Audience

Docker is used by a wide range of professionals in the tech industry. Software developers use it to build, test, and run applications in the same environment, avoiding issues where code works on one system but not another. DevOps Engineers rely on Docker for automating deployments and managing infrastructure, making continuous integration and delivery (CI/CD) more efficient. [18]

System administrators benefit from Docker as it helps them manage applications across different environments without dependency conflicts. Cloud engineers use it to deploy and scale applications seamlessly on cloud platforms like AWS, Google Cloud, and Azure.

Data scientists and AI/ML engineers package their models, libraries, and dependencies in Docker containers, making it easier to share and deploy machine learning applications. IT security professionals use Docker to create isolated environments for testing vulnerabilities and securing applications.

Finally, QA and testing teams use Docker to set up consistent testing environments, ensuring that software behaves the same way across different setups.

2.12 Security with Docker

When working with Docker, security is an important concern. Since containers share the host OS kernel, a vulnerability in one container could potentially affect others or even the host system if not properly secured. Unlike virtual machines, which are fully isolated, Docker containers rely on the same underlying system, making them more exposed to security risks. [2]

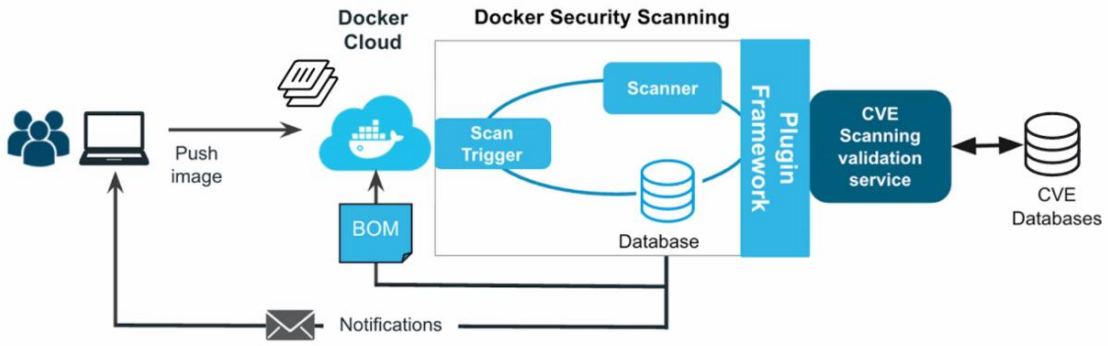


Figure 3 Docker Security Scanning

One major concern is image security. If developers use unverified or outdated images from public repositories, they might introduce vulnerabilities into their applications. In Figure 3 we show why it is always recommended to use official or trusted sources and regularly update images to include security patches. [19]

Another issue is container isolation. By default, containers run with certain privileges that can be exploited if not restricted. Users should avoid running containers as root and apply security policies like user namespaces to limit access. [17]

Network security is also a key factor. Docker's default networking settings might not be secure for production environments, so configuring firewalls, network policies, and encrypted communication is crucial. Finally, data persistence and access control should be handled carefully. If a container is compromised, an attacker could gain access to mounted volumes or sensitive files. Using proper file permissions and read-only containers can help minimize risks. To improve security, users should follow best practices like scanning images for vulnerabilities, applying the principle of least privilege, enabling Docker security features, and keeping both Docker and the host system updated. [15]

To better understand how this is extremely harmful and impactful in a company environment for users and developers, we can share a couple of examples:
Docker Escape Vulnerability (CVE-2019-5736)

Where attackers could break out of a container and gain control of the host system. This happened because of a weakness in *runc*, the low-level container runtime.

This security misconfiguration can be mitigated by always keeping the Docker version updated, using read-only file systems and running containers as non-root users, to limit the attacker's scope to only the container itself, therefore limiting potential damage to the Host OS and the user or company network.

Kubernetes Cluster Compromise Through Insecure Containers:

Like in the example above, the exploitation of Docker misconfigurations and outdated versions can create a huge amount of damage to a company by, for example, exploiting the company's network leading to privilege escalation, accessing Kubernetes nodes and possibly taking over the cluster.

This can be avoided by not using Docker containers running with too many privileges (never run containers with *--privileged* mode unless absolutely necessary), setting up network policies to limit the container-to-container communication and finally enforcing the usage of Role-Based Access Control to once again limit the user's capabilities while using Docker containers.

Other common security policies you should have in mind are for example never leaving hardcoded credentials inside Dockerfiles or in application codes, and regularly scan images for sensitive information to catch exposed credentials before the final deployment. With this, we hope to keep in mind some security principles while using Docker in a project for yourself, or even while working in university or for a company.

To conclude this chapter, we recommend that you practice using the deck of Flash Cards provided in the QR Code in the Figure 4.



Figure 4 QR Code for the deck of flash cards

3. Outlook of the technology

In this section of the work, the installation of Docker on different operating systems (Linux, macOS, and Windows) will be covered.

3.1 Setup/Installation (SW/HW)

3.1.1 Linux

In this case, the installation will be performed on Ubuntu. If you are using another distribution such as Debian or Fedora, please visit https://docs.docker.com/desktop/setup/install/linux/ (Docker, s.d.).

Update the system and install the necessary dependencies:

1. sudo apt-get update
2. sudo apt-get install ca-certificates curl

Add GPG key:

3. sudo install -m 0755 -d /etc/apt/keyrings
4. sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
5. sudo chmod a+r /etc/apt/keyrings/docker.asc

Add the Docker repository to the APT sources list:

6. echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

Update the system again and install docker:

7. sudo apt-get update
8. sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

Verify the installation:

9. docker --version

3.1.2 Windows

The most effective way to install Docker on Windows is through Docker Desktop, which requires either the WSL 2 subsystem or Hyper-V. In this case, we will use WSL 2 (Docker, s.d.).

1. Download Docker from the official website: https://docs.docker.com/desktop/setup/install/windows-install/

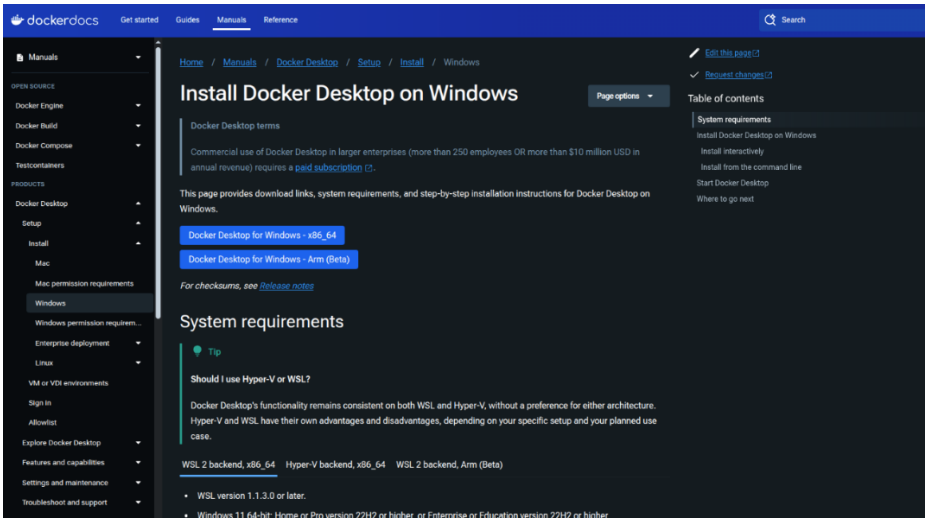


Figure 5 Download from official website

2. Run the executable.

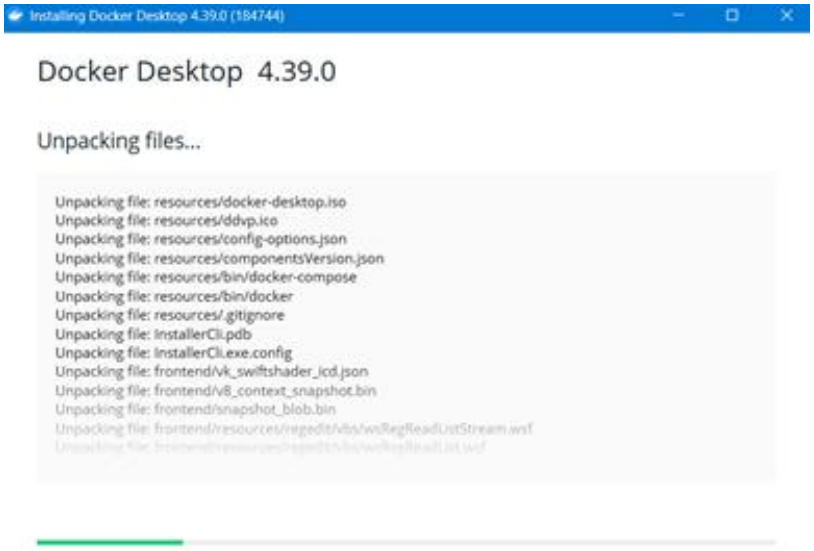


Figure 6 First interface after running the executable

3. If WSL 2 is not installed, install and activate it during the installation process.

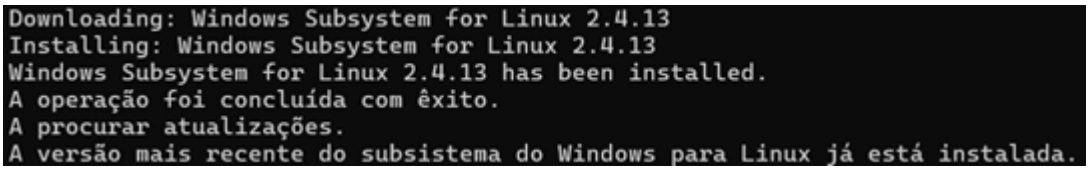


Figure 7 Install WSL

4. Start Docker Desktop.

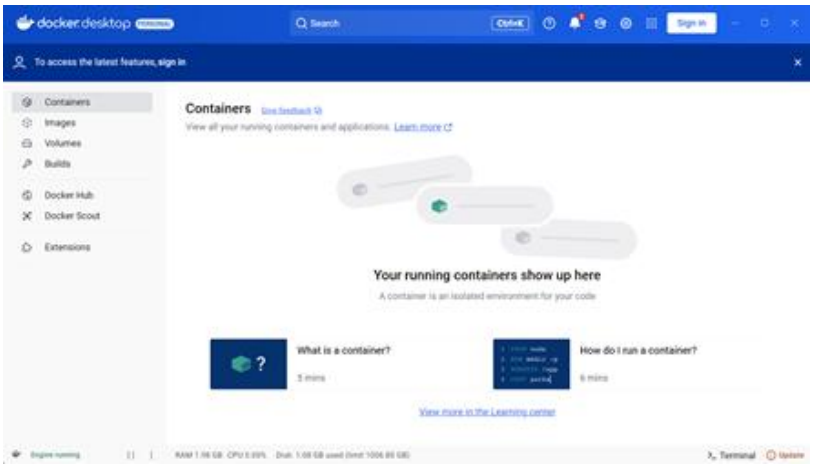


Figure 8 Interface Docker Desktop

5. Verify the installation

```
PS C:\Users\mateu> docker --version
Docker version 28.0.1, build 068a01e
```

Figure 9 Verify version of Docker

3.1.3 Mac

The most effective way to install Docker on a Mac is also through Docker Desktop (Docker, s.d.).

1. Download docker from the official website:

https://docs.docker.com/desktop/setup/install/mac-install/

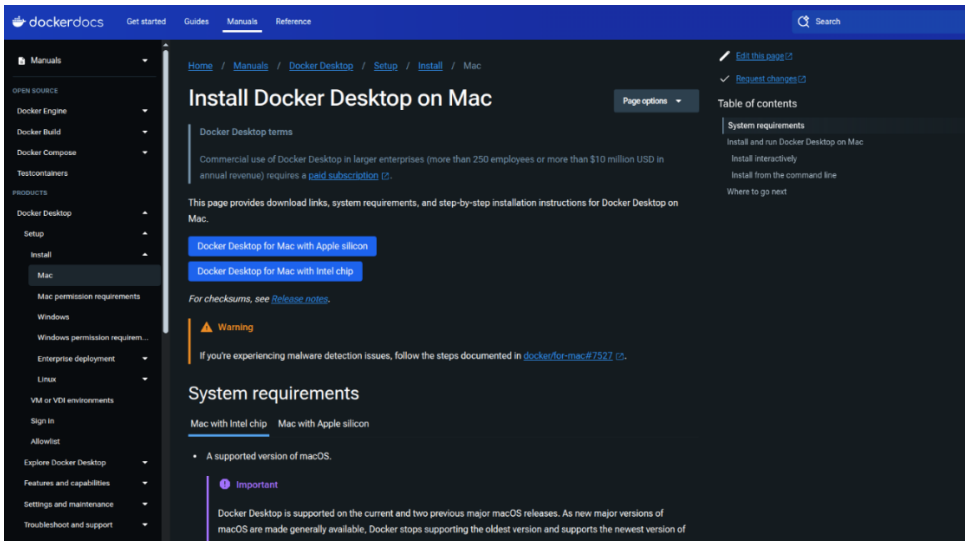


Figure 10 Download from official website

2. Open the installer and drag the icon to the applications folder.



Figure 11 Installation on Mac

3. Start Docker Desktop

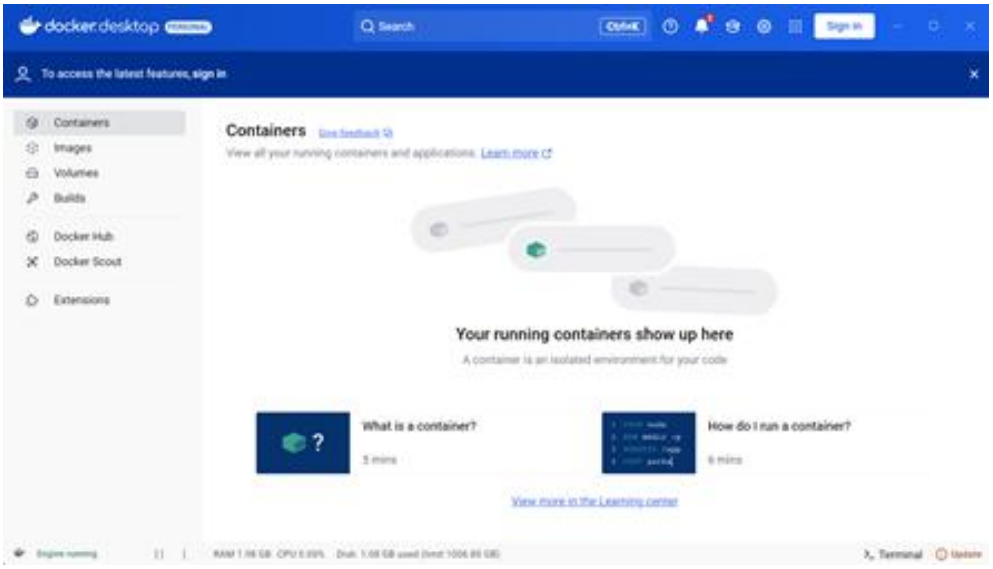


Figure 12 Interface Docker Desktop

3.2 First steps

In this chapter, the initial and fundamental Docker commands will be presented. In the following images, regarding management of container and images, as well as main commands for creating a custom Dockerfile.

```
# Images
## List Images:
docker image ls

## Create an Image:
docker image build . -t <name>
docker build . -t <name>

## Pull an Image:
docker image pull <image>

## Remove an Image:
docker image rm <image>

## Remova all Images:
docker image prune
```

Figure 13 Docker CLI image commands

```
# Containers

## List Containers:
docker container ls
docker ps
docker container ls -a (List all even if already stopped)
docker ps -a

## Run a Container:
docker container run <image>
docker run <image>
docker run -d <image> (Run detached)
docker run -t <image> (Run with tty)
docker run -it <image> (Run with tty and pass STDIN)
docker run -it <image> bash (run with tty and pass STIN, and have a bash cosole)
docker run --name <name> (Attribute a custom name to a container)

## Pause Container
docker pause <id or name>

## Unpause Container
docker unpause <id or name>

## Stop a Container:
docker container stop <id or name>
docker stop <id or name>
```

```
## Kill a Container
docker kill <id or name>

## Remove a Container:
docker container rm <id or name>

## Remova all Containers:
docker container prune

## See Logs of container
docker logs -f <id or name>

## Attach to sessions on Container
docker attach

## Execute Commands on Container
docker exec <id or name> <command>
docker exec -it <id or name> bash (Create a bash session on a container)

## Copy Files do Container
docker cp <file> <id or name>:<path>

## See diff/changes over time
docker diff <id or name>
```

Figure 14 Docker CLI containers commands

If we intend to create a custom docker image, we will need to create a file called Dockerfile and set some custom commands inside of it that allow us to do some operations regarding the container. Below we can see some of the most used commands.

```
# Dockerfile Basic Commands

## 1. Specify Base Image

FROM ubuntu:latest  # Use an official Ubuntu base image

## 2. Set Maintainer Information

LABEL maintainer="your-email@example.com"

## 3. Set Environment Variables

ENV APP_HOME=/app

## 4. Set Working Directory

WORKDIR $APP_HOME

## 5. Copy Files to Container

COPY . /app

## 6. Add Files from URL

ADD https://example.com/file.tar.gz /tmp/


## 7. Run Commands During Build

RUN apt-get update && apt-get install -y curl

## 8. Expose Ports

EXPOSE 80

## 9. Define Startup Command

CMD ["python", "app.py"]

## 10. Define Alternative Entry Point

ENTRYPOINT ["/bin/bash"]
```

```
## 11. Use Arguments for Dynamic Values

ARG VERSION=1.0

## 12. Set User Permissions

USER root

## 13. Define Volume for Persistence

VOLUME ["/data"]

## 14. Stop Container Gracefully

STOPSIGNAL SIGTERM
```

Figure 15 Basic Dockerfile commands

4. Tutorial/Functionality

4.1 Docker Compose / Multi Container APPs

Docker Compose is a crucial tool for modern software development, designed to simplify the management of multi-container Docker applications. It acts as an orchestration layer on top of the Docker Engine, allowing developers to define and configure an entire application stack within a single, declarative docker-compose.yml file.

This file serves as a blueprint, outlining all the individual containers, their interconnections, and resource requirements. Instead of managing numerous containers individually using the Docker CLI, Docker Compose provides a unified approach to initiate, terminate, and oversee complex applications as a single entity, significantly easing the management burden.

4.2 Structure of a docker-compose.yml File

A typical docker-compose.yml file begins with an optional version field, which specifies the Docker Compose file format version being used. This version can influence the available features and syntax, so it's important to ensure compatibility with the installed Docker Compose version.

The main body of the file is organized into top-level elements, primarily including services, networks, and volumes.

The services section is where individual containers, or services, are defined. Each service definition includes various directives that specify how the container should be run, such as the Docker image to use, ports to expose, environment variables to set, and dependencies on other services. The networks section allows for the definition of custom networks that the services can be attached to, providing isolated communication channels. Finally, the volumes section is used to define named volumes for persistent data storage.

A basic example of a docker-compose.yml file might look like this:

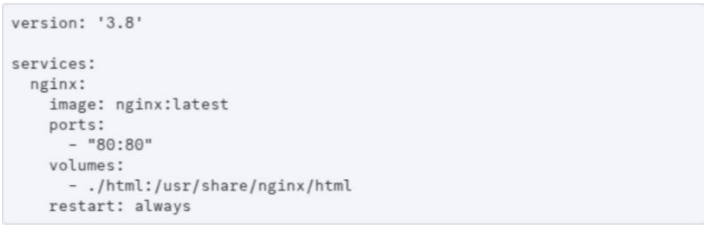


Figure 16 Template Docker Compose.yml

4.3 Advantages of Docker Compose

Docker Compose offers significant advantages by simplifying the orchestration of multi-container applications through a single configuration file. It effectively manages dependencies between services, ensuring they start in the correct order. Development workflows are greatly improved by enabling the quick setup of consistent and reproducible environments. While primarily for development and testing, it provides basic scalability for local simulations. Finally, Docker Compose contributes to cost-effectiveness by leveraging existing resources and simplifying operational overhead.

Feature	Description
Simplified Orchestration	Manages multiple containers as one unit with simple commands from a single configuration file
Dependency Management	Ensures services start in the correct order using depends_on, with health checks recommended for full readiness
Improved Development Workflows	Enables quick, easy, and consistent setup of development environments, boosting productivity
Scalability (Dev & Test)	Provides basic scaling for local development and testing via the scale command, but is limited to a single host
Cost-Effective	Reduces costs by reusing images, simplifying operations, and allowing for efficient resource utilization

Figure 17 Docker Compose advantages

4.4 Docker hub

Docker Hub is a Docker image repository online where secure and effective storage and distribution of containers may be accomplished. Public and private images may also be stored on the repository.

This tool is also very important to development teams, as it enables them to share containerized applications and therefore improve collaboration (Docker, s.d.).

4.5 Docker CLI

The Docker CLI is the command-line interface used to interact with the Docker Engine. With it, you can manage containers, images, and other Docker configurations quickly and efficiently, such as creating and starting containers without the need for a graphical interface (Docker, s.d.).

Some of the main commands provided by the Docker CLI have already been presented earlier in this report.

4.6 Docker desktop

Docker desktop is a graphical interface that simplifies the use of Docker. With this interface, users can access an environment that includes the Docker CLI, Docker Compose, and Docker Hub, making it easier to run containers.

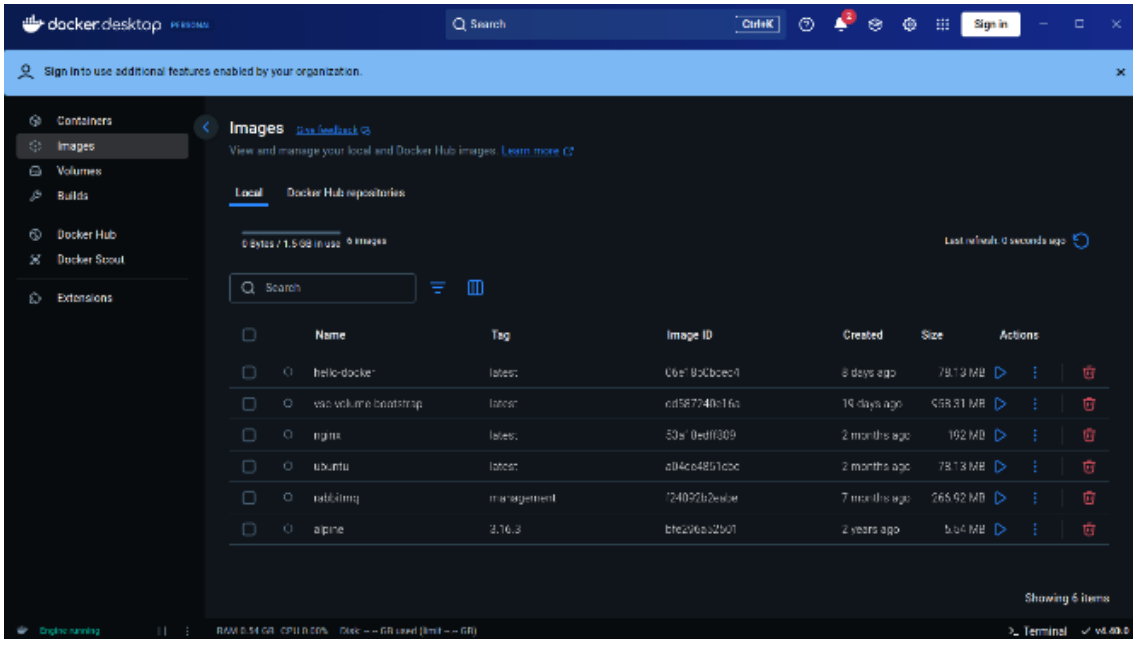


Figure 18 Interface Docker Desktop

5. Exercise(s)

5.1 Problem A

Containerization has become a crucial aspect of modern software deployment. However, managing those same containers through the command line interface (CLI) and dealing with configuration files like YAML or Kubernetes manifests can be complex and prone to errors, especially for people who are beginners in networking or containerization itself. To address this current issue, this problem explores the use

and deployment of **Portainer** which provides a web-based graphical user interface (GUI) that abstracts the complexity and technical knowledge of Docker and its configuration syntax. The following problem demonstrates how to install and run Portainer CE (Community Edition) on a local Docker environment, showing how it addresses the issue of usability and accessibility in container orchestration.

5.1.1 (Re)solution A

The first step is to create a volume:

```
~$ docker volume create portainer-volume
portainer-volume
```

After that we pull the image:

```
~$ docker pull portainer/portainer-ce:lts
```

You should see something like this:

```
lts: Pulling from portainer/portainer-ce
6970b305841e: Pull complete
c05d4fb47930: Pull complete
04de093ad5ed: Pull complete
a9ff7abff372: Pull complete
e09df2601140: Pull complete
df254cde32a0: Pull complete
6b34a21056c4: Pull complete
faf3a09a336d: Pull complete
26693e01e9e2: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:449202d765d28ec443c1657fc1121aff92b8afcee6b58bcea36e1f0e81e
8297c
Status: Downloaded newer image for portainer/portainer-ce:lts
docker.io/portainer/portainer-ce:lts
What's next:
View a summary of image vulnerabilities and recommendations → docker scout
quickview portainer/portainer-ce:lts
```

Then run the Portainer in a container with the appropriate settings:

```
~$ docker run -d \
-p 8000:8000 \
-p 9443:9443 \
--name portainer \
--restart=always \
-v /var/run/docker.sock:/var/run/docker.sock \
-v portainer-volume:/data \
portainer/portainer-ce:lts
17f87732d533f484aafabeed5d577096b7659c0419ac043d9e5c204d58e2222b
```

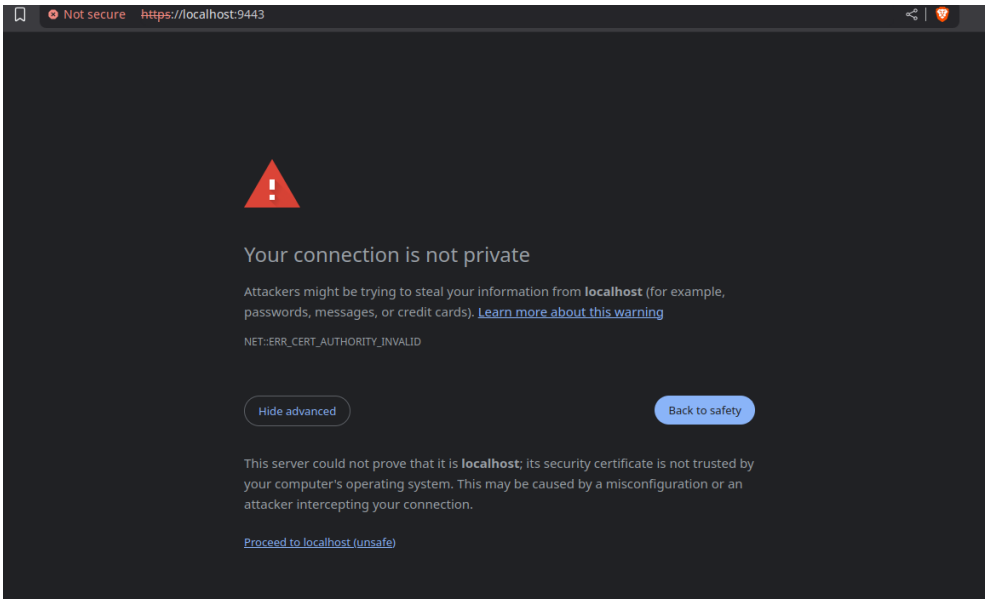


Figure 19 Docker container in port 9443

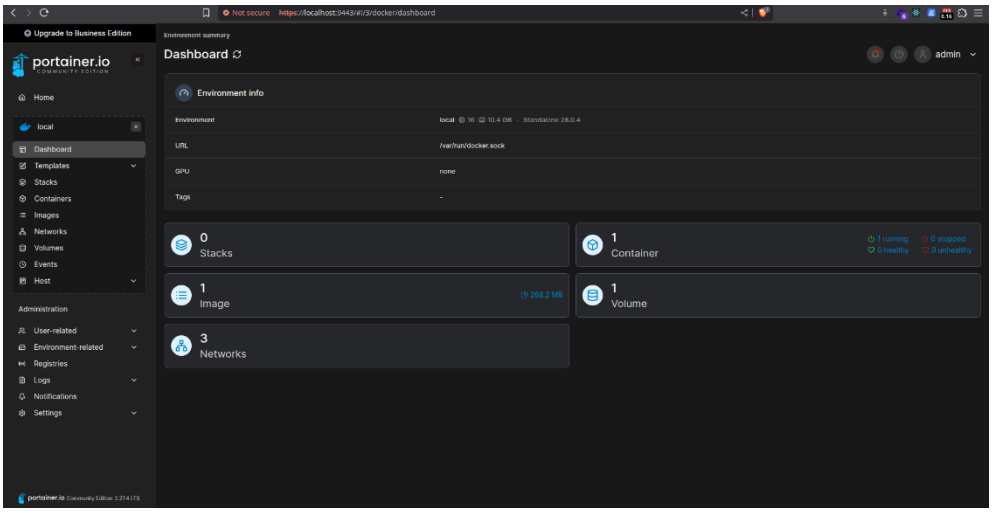


Figure 20 Portainer UI

See the containers that are running:

```
~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
17f87732d533 portainer/portainer-ce:lts "/portainer" 6 minutes ago Up 6 minutes
0.0.0.0:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, 9000/tcp portainer
Stop the container:
~$ docker stop portainer
Portainer
```

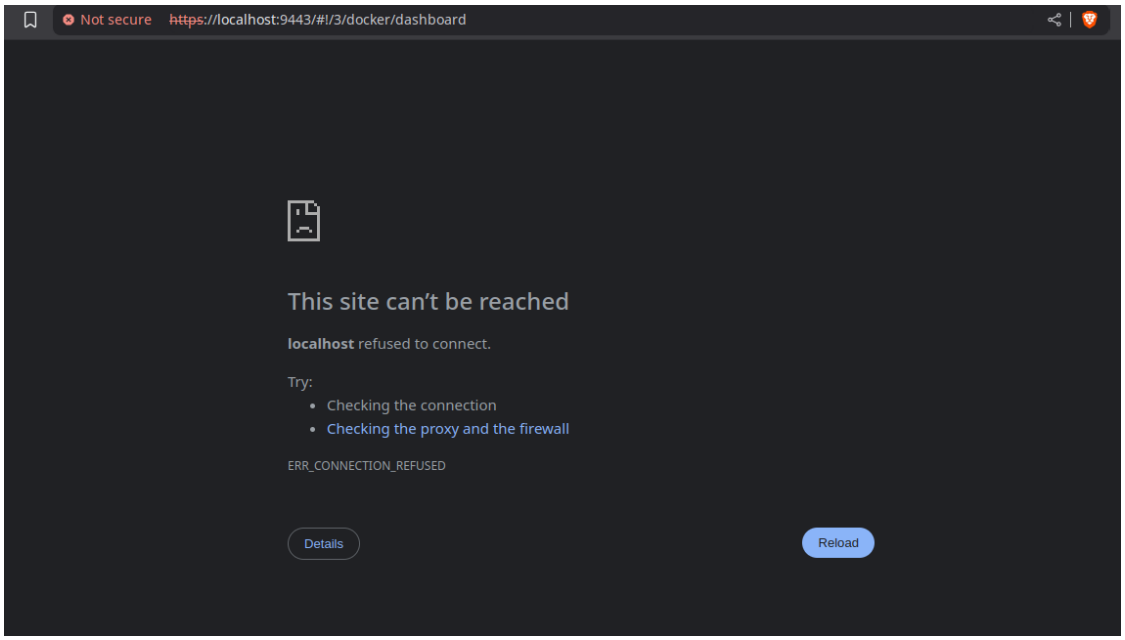


Figure 21 Docker Dashboard

```
~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

If you add the flag –a you can see all the containers, including the ones not running:

```
~$ docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
17f87732d533 portainer/portainer-ce:lts "/portainer" 8 minutes ago Exited (2)
About a minute ago portainer
```

To finalize, you can now remove all the components:

```
~$ docker rm portainer
Portainer
~$ docker rmi portainer/portainer-ce:lts
Untagged: portainer/portainer-ce:lts
Untagged:portainer/portainer-
ce@sha256:449202d765d28ec443c1657fc1121aff92b8afcee6b58bcea36e1f0e81e8297c
Deleted:sha256:30d09e5cd7971815bd277ab897bd254a5c8222971be4b00e1f0bc8f301d508
90
Deleted:sha256:7f43420b00b59c3c0ea894c5820350fab4f98ea28da6d102efa33b02cf4543
28
Deleted:sha256:ba04b94241d6f1a8bb289161174fcef8df80172a21fcf7e77195ae390d8f33
b2
Deleted:sha256:7e1a7ac1e915e918764b1b435596666de183c92518fb2443f4593d2941e5f7
37
Deleted:sha256:de7c48bdd88522343a5665625177f9ddce8de560c53f88c5bdf26a1ade319d
c8
Deleted:sha256:ceb0cb6fc811cb011308ea1a9bbad9b60bc5ffb0668e2e0914de6da99e73b4
da
Deleted:sha256:a75ea41e346b590c03615954c4f71bafc3b7eebb27142135a6676cbc150f53
63
Deleted:sha256:70393ec26524deef3734071822e442c80ea791a35abf8ee49a94800a37fb11
90
```

```
Deleted:sha256:4aaef52d4076db7e999ee0d38b6e9fe4b2c5f726ed749dcec945a576238745
f3
Deleted:sha256:87e33bbf1282dc21bfa035cc666b41aede45c46989674a23b2d2154480c1ea
50
Deleted:sha256:c2f85eb1031bf04865230f757fa742c9a7699c9203cb630f46d91ddc8d95d8
02
~$ docker volume rm portainer-volume
portainer-volume
```

5.2 Problem B

The goal of this problem is to build a user interface (UI) using **Appsmith**, a low-code platform, and simulate a REST API using **JSON Server**. The **clue/json-server** image is used to expose port **3000** on the host, mapping to port **80** inside the container, with a db.json file mounted as a volume to serve the API data. The UI is developed in **Appsmith**, running with the official **appsmith/appsmith-ce** image and accessible through port **8080** on the host. Appsmith connects to the REST API running in another container, demonstrating integration between containerized services and low-code applications.

5.2.1 (Re)solution B

The first step is to create the docker-compose.yml: The file shown in (Figure 22) is a representation of the docker-compose.yml file which u can use to complete problem B (Docker, s.d.).

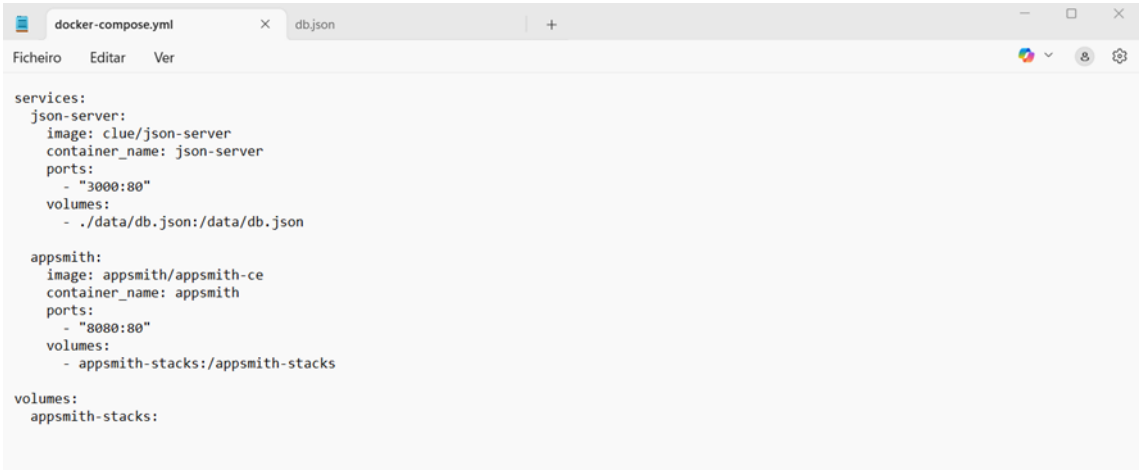


Figure 22 docker-compose.yml

The db.json file in image (Figure 23) shows an example of 2 entries in the database which will be used to show the usage of Appsmith (Docker, s.d.):

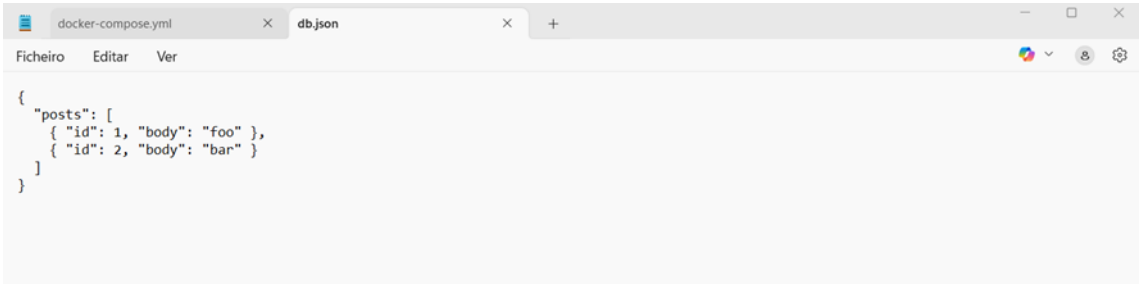


Figure 23 db.json

Start Containers (Figure 24):

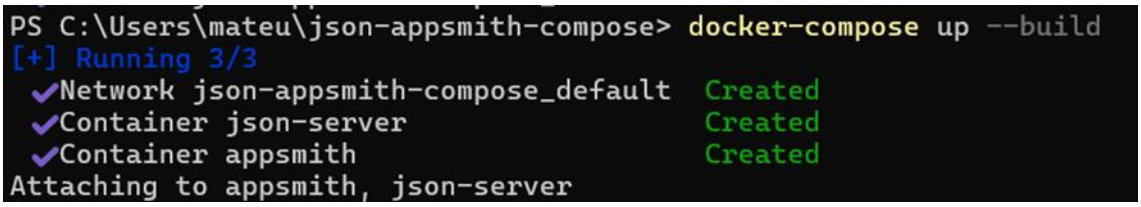


Figure 24 Start containers

After building the docker-compose with the json-server and Appsmith containers, we have access to the entries we put in the db.json file in *localhost:3000/posts*, as shown in image (Figure 25).



Figure 25 Access localhost:3000/posts

To finalize the problem, you just need to complete the setup in Appsmith *localhost:8080/setup/welcome* and test the app in Appsmith as shown in the two figures below:

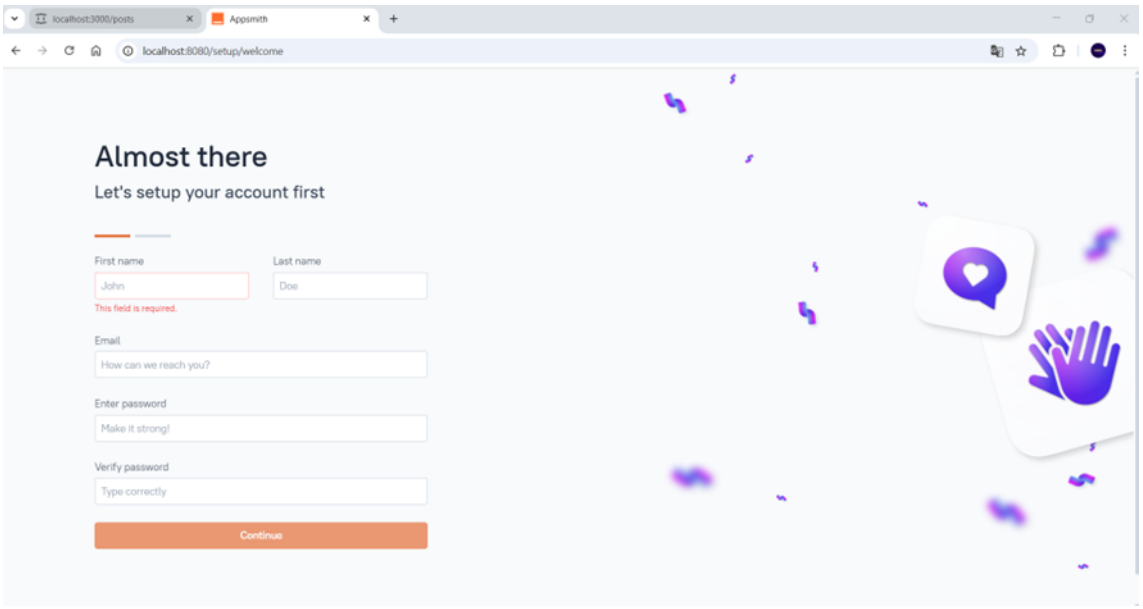


Figure 26 Access Appsmith

Now inside Appsmith itself we can use a GET request in *http://json-server:80/posts* to access the entries that were used in the db.json file (Figure 27):

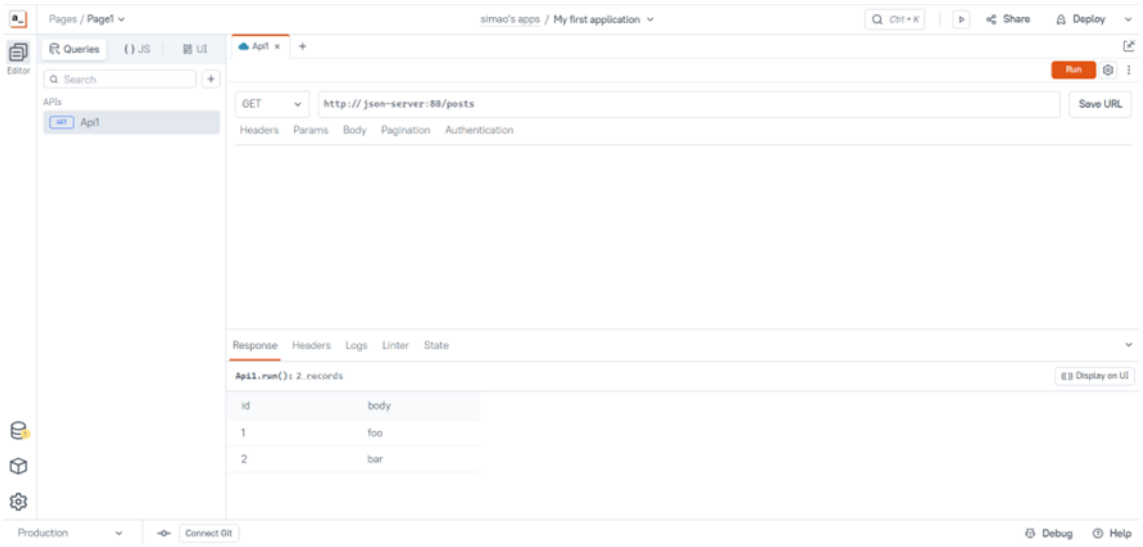


Figure 27 API request in Appsmith

6. Challenge

The objective of this challenge is to create a Flask app in a docker container using the repository https://github.com/digitalocean/sample-flask. For that, create a DockerFile, in order to create your own docker image, and then run the container. You can go to docker hub and try to find the best docker image and use it as the

start point. Then you need to copy the files into the docker image, install the requeriments.txt and then find a way to run the app not as a root user, and only with the necessary permissions (you can create a new user). Try to implement good practices. Some examples could be not keeping cache every time it is possible, not running the container as root for security purposes and trying to use a smaller docker image from docker hub like python image with alpine. You could also take a look at the 12-factor app available on the internet in order to see some good practices.

7. References

[1] Buchanan, I. (s.d.). Atlassian. Obtido em 27 de 03 de 2025, de Containers vs. virtual machines: https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms

[2] Clancy, M. (14 de 07 de 2023). What's the Diff: VMs vs. Containers. Obtido em 26 de 03 de 2025, de Backblaze: https://www.backblaze.com/blog/vm-vs-containers/

[3] Docker. (s.d.). clue/json-server. Obtido em 15 de 04 de 2025, de Docker Hub: https://hub.docker.com/r/clue/json-server

[4] Docker. (s.d.). Docker. Obtido em 22 de 04 de 2025, de Docker: https://docs.appsmith.com/getting-started/setup/installation-guides/docker

[5] Docker. (s.d.). Docker CLI. Obtido em 29 de 03 de 2025, de Docker: https://www.docker.com/products/cli/

[6] Docker. (s.d.). Docker Engine security. Obtido em 29 de 03 de 2025, de Docker Docs: https://docs.docker.com/engine/security/

[7] Docker. (s.d.). Docker Hub. Obtido em 30 de 03 de 2025, de Docker: https://www.docker.com/products/docker-hub/

[8] Docker. (s.d.). Install Docker Desktop on Mac. Obtido em 29 de 03 de 2025, de Docker: https://docs.docker.com/desktop/setup/install/mac-install/

[9] Docker. (s.d.). Install Docker Desktop on Windows. Obtido em 30 de 03 de 2025, de Docker Docs: https://docs.docker.com/desktop/setup/install/windows-install/

[10] Docker. (s.d.). Install Docker Engine on Ubuntu. Obtido em 30 de 03 de 2025, de Docker Docs: https://docs.docker.com/engine/install/ubuntu/

[11] Docker. (s.d.). Security. Obtido em 30 de 03 de 2025, de Docker Docs: https://docs.docker.com/security/

[12] Docker. (s.d.). What is a registry? Obtido em 01 de 04 de 2025, de Docker Docs: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-registry/

[13] Docker. (s.d.). What is an image? Obtido em 01 de 04 de 2025, de Docker Docs: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/

[14] Docker. (s.d.). What is Docker? Obtido em 24 de 03 de 2025, de Docker Docs: https://docs.docker.com/get-started/docker-overview/

[15] Owasp. (29 de 03 de 2025). Docker Security Cheat Sheet. Obtido de Owasp Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

[16] Team, E. Y. (18 de 03 de 2022). Docker Vs Virtual Machines Explained. Obtido em 26 de 03 de 2025, de Engine Yard: https://www.engineyard.com/blog/docker-vs-virtual-machines-explained

[17] Tigera. (s.d.). Docker Security: 5 Risks and 5 Best Practices for Securing Your Containers. Obtido em 30 de 03 de 2025, de Tigera: https://www.tigera.io/learn/guides/container-security-best-practices/docker-security/

[18] Wikipédia. (23 de 04 de 2025). Docker (software). Obtido em 26 de 03 de 2025, de Wikipédia: https://en.wikipedia.org/wiki/Docker_%28software%29

[19] Yi, K. (s.d.). Container Security and Why It Matters. Obtido em 30 de 03 de 2025, de Docker: https://www.docker.com/blog/container-security-and-why-it-matters/